

Alternatively we could use path patching to confirm the precise interactions. Say we want to test whether the Peace embedding is necessary as an input to L0H0, as an input to L1N42, or both. For this we could patch only the corresponding paths, and find that denoising (1) “Nobel $\rightarrow$ L0H0” and (2) “Peace $\rightarrow$ L1N42” paths is sufficient. Alternatively we might find that noising every path except for (1) “Nobel $\rightarrow$ L0H0”, (2) “L0 $\rightarrow$ L1N42”, and (3) “Peace $\rightarrow$ L1N42” does not break performance. Note again that denoising only required restoring two paths (restoring a cross-section of the circuit) while noising required leaving 3 paths clean (the full circuit).⁴

3.2 Concepts & gotchas

The walkthrough above presents a typical circuit discovery workflow. We want to highlight a couple of additional concepts and common issues.

Sensitivity & prompt choice: A positive patching result implies you have found activations dealing with the *difference between the clean and corrupt prompt*. Make sure to consider all degrees of freedom in a task, and consider multiple sets of corrupted prompts if necessary.

Scope of activation patching: More generally, activation patching is always based on prompt distributions, and does not make statements for model behaviour outside these specific distributions. For more discussion on the limitations of patching, and the specificity of prompt-based interpretability in general, see Neel Nanda’s writing on What Can(’t) Activation Patching Teach Us.

No minimality: Here, and in many parts of the literature, a circuit is treated as a collection of model components that are responsible for a particular model behaviour. We typically make no claims that we have found the *smallest* such collection of components, we only test that this collection is sufficient.

Backup behaviour & OR-gates: In some cases researchers have discovered “Backup heads”, components that are not normally doing the task but jump into action if other components are disrupted (Hydra effect, McGrath et al., 2023). For example, in IOI when one ablates a name mover head (a key component of the circuit) a backup name mover head will activate and then do the task instead (Wang et al., 2022).

It can be helpful to think of these as OR-gates where either component is sufficient for the model to work. This does not fit well into our attempts of defining a circuit, nor plays well with the circuit finding methods above. Despite the name mover heads being important, if we ablate them then, due to backup heads compensating, the name movers look less important. Fortunately, backup behaviour seems to be lossy, i.e. if the original component boosted the logits by $+X$, the backup compensates for this by boosting less than X (the Hydra effect paper found $0.7 \cdot X$). Thus these backup component weaken the visibility of the original component, but it is usually still visible since even $0.3 \cdot X$ is a relatively large effect.

Negative components: Some work in this area (e.g. Wang et al., 2022; Heimersheim and Janiak, 2023) noticed attention heads that consistently negatively affected performance, and noising them would increase performance. This is problematic, because it makes it hard to judge the quality of a circuit analysis: it may look like we’ve fully recovered (or more than fully recovered!) performance, by finding half the positive components but excluding all negative ones. This is an unsolved problem. Conmy et al. (2023) propose using Kullback Leibler (KL) divergence as a metric to address this, which penalises any deviation (positive or negative), at the cost of also tracking lots of variation we may not care about.

⁴This method doesn’t yet confirm which information is carried in the different paths. We can go a step further and noise (corrupt $\rightarrow$ clean patch) even some of the important circuit connections, namely “Nobel $\rightarrow$ L0H0 $\rightarrow$ L1N42” path from the “Nobel Y” distribution, and the “Peace $\rightarrow$ L1N42” path from the “X Peace” distribution. Doing that is essentially Causal Scrubbing (Chan et al., 2022).

4 Metrics and common pitfalls

So far we talked about “preserving” and “restoring” performance, but in practice, model performance is not binary but a scale. Typically we find some components matter a lot, while others provide a small increase in performance. For the best interpretability we might look for a circuit restoring e.g. 90% of the model’s performance, rather than reaching exactly 100% (for examples see Chan et al., 2022). A useful framing is the “pareto frontier” of circuit size vs. performance recovered - recovering 80% of performance with 1% of the components is more impressive than 90% of the performance with 10% of the components, but there will always be a minimum circuit size to recover a given level of performance.

It’s easy to treat metrics as an after-thought, but we believe that the right or wrong choice of a metric can significantly change the interpretation of patching results. Especially for exploratory patching, the wrong metric can be misleading. The choice of metric matters less for confirmatory patching, where you expect a binary-ish answer (“have I found the circuit or not”) and all metrics should agree. We’ll go through a couple of metric choices in this section:

Based on	Example
Logit difference (= Logprob difference)	Logit(Mary) - Logit(John)
Logarithmic probability (logsoftmax)	Logprob(Mary)
Probability (softmax)	Prob(Mary)
Accuracy / Rank of correct answer	Rank(Mary)==0

An honourable mention goes to the KL divergence. Unlike the previous metrics, this metric aims to compare the full model output, rather than focusing on a specific task. KL divergence is a good metric in such cases.

In addition to these output based metrics, in some cases it makes sense to consider some model internals as metrics themselves. For example, one might use the attention paid by the name mover head to the indirect object as a metric to identify the subcircuit controlling this head, or the activation of a key neuron or SAE feature, or the projection onto a probe (Nanda et al., 2023).

In our experience, it’s worth implementing many metrics and briefly analysing all of them. Computing a metric is cheap (compared to the cost of the forward pass), and they all have different strengths and weaknesses, and can illuminate different parts of the big picture. And if they all agree that’s stronger evidence than any metric on its own. Where they disagree, we personally trust **logit difference** (or equivalently logprob difference) the most.

4.1 The logit difference

Logit difference measures to what extent the model knows the correct answer, and it allows us to be specific: We can control for things we don’t want to measure (e.g. components that boost both, Mary and John, in the IOI example) by choosing the right logits to compare (e.g. Mary vs John, or multiple-choice answers). The metric also is a mostly linear function of the residual stream (unlike probability-based metrics) which makes it easy to directly attribute logit difference to individual components (“direct logit attribution”, “logit lens”). It’s also a “softer” metric, allowing us to see partial effects on the model even if they don’t change the rank of the output tokens (unlike e.g. accuracy), which is crucial for exploratory patching. We discuss problems with this and other metrics in the next section.

Intuition for why logits and logit differences (LDs) are a natural unit for transformers: The residual stream and output of a transformer is a sum of components. Every component added to the residual stream corresponds to an addition to the LD (as the LD corresponds to a residual stream direction, up to layer norm). A model component can easily change the LD by some absolute amount (e.g. +1 LD). It cannot easily change the LD by a relative amount ($LD \times 1.5$), or change the probabilities by a specific amount (prob += 0.20). For example consider a model component that always outputs -1 logit to duplicated names (assume “John and Mary ... John ...”). This component then always writes +1 LD in favour of Mary, and gets a score of 1 in terms of LD. Other metrics (such as probability) judge this component differently, depending on what the baseline was (e.g. due to other patches). We would argue that logits and logit differences are closer to the mechanistic process happening in the

transformer, and thus feel like a more natural unit. This is of course not a requirement, and also does not hold in all places (e.g. if a component's output depends on the input LD), but it seems to work well in practice.

4.2 Flaws & advantages of different metrics

It is essential to be aware of what a metric measures and is sensitive to. A key thing to track is whether the metric is discrete vs continuous, and whether it's exponential vs linear (in the logits) - continuous, linear metrics are usually more accurate, which is crucial when doing exploratory patching and assigning "partial credit" to model components. Here we list common pitfalls of popular metrics.

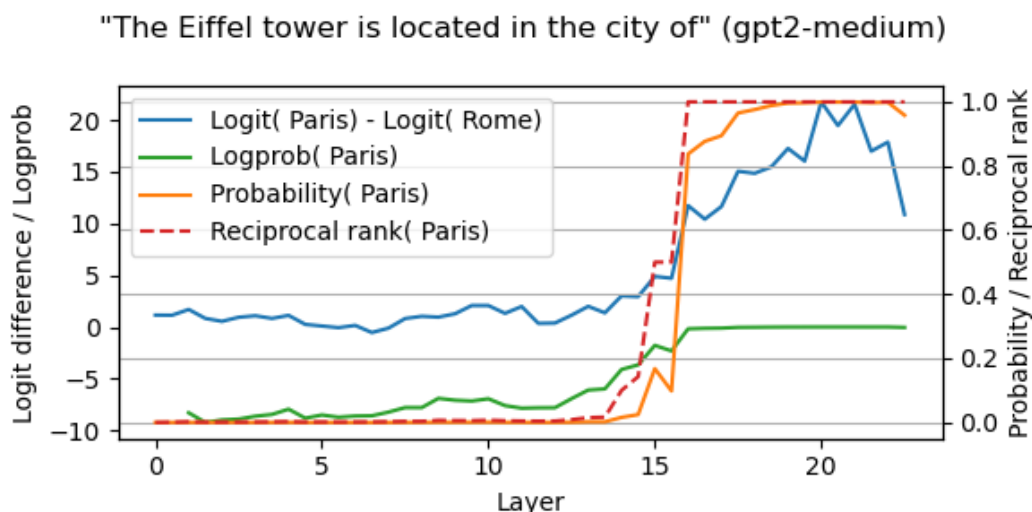


Figure 2: Illustration of different metrics for an example patching experiment with GPT-2 medium.

- **Logit difference / logprob difference:** The difference between the logit of the correct answer, and the incorrect answer(s). This metric specifically measures the difference between the selected logits, and is not sensitive to components which affect all of them. For example, in IOI it measures the model's confidence in Mary vs John which encapsulates the IOI-circuit well without being sensitive to the "is the next token a name?"-circuit.
- **Potential false-positive:** Because the metric is a *difference* it may be driven by either getting better at the correct answer or worse at the incorrect answer. Thus it is worth checking the logits or logprobs of individual answers to confirm.
This is particularly concerning because the corrupted model likely puts a high probability on the incorrect answer. This means that any patch that indiscriminately damages the model and gets it closer to uniform will damage the incorrect answer logprob and so boost the logit diff.
- **Logprobs:** This metric measures the logprob of the correct answer. It is sensitive to absolute change in logarithmic probabilities (i.e. relative change in probabilities) and captures our intuition for what good model performance means. We broadly think it is a good metric. It's main flaws are
 - **Saturation:** Once the correct answer becomes the model's top guess, the logprob stops increasing meaningfully, even though the confidence can increase much more. We can see this in Figure 2, where the green line saturates after layer 17.
 - **Unspecificity:** We lose the ability to control for other properties, e.g. in IOI we cannot distinguish between components that increase both $P(\text{John})$ and $P(\text{Mary})$ from components that only increase $P(\text{Mary})$. This can be intended, or unintended, it's just important to keep in mind.
 - **Inhibition:** To increase the logprob on John, the model can either increase the John logit, or decrease other top logits, and it is hard to distinguish which is happening. This